

UNISENAI

**CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS**

THIAGO SOETHE PASCHOAL

MÓDULO 0 - AMBIENTE PROFISSIONAL DE DESENVOLVIMENTO NODE.JS

Atividade teórica - pesquisa e fundamentação

Disciplina: Node.js - Projeto Guiado

Prof. Carlos Uchôa

Prof. William Cesar Sestito Ribeiro

Joinville

2026

1 INTRODUÇÃO

O desenvolvimento com Node.js envolve mais do que escrever código. Antes de iniciar um projeto, é necessário entender como a plataforma executa tarefas, como o JavaScript trabalha com operações assíncronas e como ferramentas como TypeScript, NVM e Docker ajudam a organizar o ambiente. Para quem está começando em Análise e Desenvolvimento de Sistemas, esses conceitos são importantes porque aparecem em projetos web, APIs e aplicações que precisam funcionar de forma parecida em diferentes computadores. Este trabalho apresenta os principais pontos solicitados no Módulo 0 e relaciona cada conceito a situações simples de desenvolvimento. (UCHÔA; SESTITO, 2026)

2 DESENVOLVIMENTO

2.1 Event loop do Node.js

O event loop é o mecanismo que organiza a execução de tarefas assíncronas no Node.js. A thread principal continua disponível enquanto operações demoradas, como acesso a arquivos, banco de dados ou rede, são tratadas pelo ambiente. Quando uma dessas operações termina, sua função de retorno entra na fila para ser executada. Isso permite que uma aplicação atenda muitas conexões sem precisar criar uma thread principal para cada usuário. Um exemplo é uma API de agendamentos: enquanto uma consulta ao banco está sendo processada, o servidor pode continuar recebendo outras requisições. (NODE.JS FOUNDATION, 2026)

2.2 Operações síncronas e assíncronas

Uma operação síncrona precisa terminar antes que o programa continue para a próxima etapa. Já uma operação assíncrona permite que o restante do código continue enquanto o resultado é aguardado. Em Node.js, bloquear o event loop com uma tarefa pesada pode atrasar todas as outras requisições do servidor. Por exemplo, se uma API fizer um processamento grande de forma síncrona antes de responder, outros usuários podem ficar esperando. Por isso, operações de entrada e saída normalmente são tratadas de forma assíncrona, usando callbacks, Promises ou async/await. (UCHÔA; SESTITO, 2026)

2.3 TypeScript e JavaScript

TypeScript é uma extensão do JavaScript que acrescenta tipagem estática durante o desenvolvimento. No JavaScript puro, uma variável pode receber tipos diferentes e muitos erros só aparecem quando o programa está rodando. No TypeScript, o compilador consegue avisar antes da execução quando um valor não corresponde ao tipo esperado. Por exemplo, em uma função que recebe a idade de um paciente como número, passar um texto pode ser identificado ainda no editor ou na compilação. Depois da verificação, o código TypeScript é transformado em JavaScript para ser executado pelo Node.js. (UCHÔA; SESTITO, 2026)

2.4 CommonJS e ES Modules

CommonJS é o formato tradicional de módulos do Node.js e utiliza `require` para importar e `module.exports` para exportar. ES Modules utiliza `import` e `export`, seguindo o padrão moderno do JavaScript. Os dois formatos podem funcionar em projetos Node.js, mas a mistura entre eles pode gerar problemas de compatibilidade, principalmente com bibliotecas que suportam apenas um modelo. Em um projeto atual, o uso de ES Modules costuma ser indicado quando o `package.json` está configurado com `"type": "module"`. O importante é escolher um padrão e manter o projeto consistente. (UCHÔA; SESTITO, 2026)

2.5 Frameworks web: Express, Fastify e NestJS

Express é um framework simples e flexível para criar APIs e aplicações web. Ele não obriga o desenvolvedor a seguir uma estrutura grande, o que facilita o aprendizado de rotas, middlewares e respostas HTTP. Eu escolheria Express para uma API pequena ou para um projeto acadêmico em que o objetivo seja entender primeiro os fundamentos do backend. (UCHÔA; SESTITO, 2026)

Fastify também é voltado para APIs, mas possui foco maior em desempenho e oferece recursos de validação por schema. Ele pode ser interessante em um serviço que recebe muitas requisições e precisa manter uma estrutura relativamente simples. Um exemplo seria uma API de consultas rápidas, na qual tempo de resposta e validação dos dados são importantes. (UCHÔA; SESTITO, 2026)

NestJS é mais estruturado e organiza a aplicação em módulos, controllers, services e providers. Essa organização facilita o trabalho em sistemas maiores,

principalmente quando vários desenvolvedores trabalham no mesmo projeto. Eu escolheria NestJS para um sistema de clínica ou gestão empresarial com muitos módulos, regras de negócio e integrações, porque a estrutura ajuda a manter o código organizado conforme o projeto cresce. (NESTJS, 2026)

2.6 Gerenciador de versões do Node.js (NVM)

O NVM é uma ferramenta usada para instalar e alternar entre versões diferentes do Node.js na mesma máquina. Isso é importante porque nem todos os projetos usam a mesma versão. Um sistema antigo pode depender de uma versão anterior, enquanto um projeto novo utiliza a versão LTS atual. Sem um gerenciador, seria necessário desinstalar e reinstalar o Node.js para trocar de versão. Em times profissionais, o NVM reduz esse problema e ajuda cada integrante a trabalhar com a versão definida pelo projeto, inclusive com o uso de um arquivo `.nvmrc`. (NVM-WINDOWS, 2026)

2.7 Container Docker e máquina virtual

Um container Docker executa uma aplicação de forma isolada usando uma imagem que contém o ambiente e as dependências necessárias. Ele compartilha recursos do sistema hospedeiro e tende a iniciar rapidamente. Uma máquina virtual, por outro lado, executa um sistema operacional completo sobre uma camada de virtualização, consumindo mais memória e armazenamento. Por exemplo, para testar uma API Node.js, um container pode carregar apenas Linux, Node.js, dependências e a aplicação, enquanto uma máquina virtual precisaria manter um sistema operacional completo. Containers e máquinas virtuais resolvem problemas diferentes e podem inclusive ser usados juntos. (DOCKER INC., 2026)

3 CONCLUSÃO

Os conceitos estudados no Módulo 0 formam a base para trabalhar com Node.js de maneira mais organizada. O event loop e a programação assíncrona ajudam a entender o funcionamento do servidor; TypeScript melhora a identificação de erros durante o desenvolvimento; os módulos e frameworks definem como o código pode ser estruturado; o NVM facilita o controle de versões; e o Docker permite executar a aplicação em um ambiente padronizado. Para um estudante de ADS, compreender essas ferramentas desde o início facilita as próximas atividades e aproxima o desenvolvimento acadêmico das práticas usadas em projetos reais.

REFERÊNCIAS

DOCKER INC. Docker documentation. Disponível em: <<https://docs.docker.com>>. Acesso em: 20 ago. 2026.

NESTJS. NestJS documentation. Disponível em: <<https://docs.nestjs.com>>. Acesso em: 20 ago. 2026.

NODE.JS FOUNDATION. Node.js documentation. Disponível em: <<https://nodejs.org/en/docs>>. Acesso em: 20 ago. 2026.

NVM-WINDOWS. nvm-windows documentation. Disponível em: <<https://github.com/coreybutler/nvm-windows>>. Acesso em: 20 ago. 2026.

UCHÔA, Carlos; RIBEIRO, William Cesar Sestito. Node.js - Projeto Guiado: Módulo 0 - Montando o Ambiente Profissional de Desenvolvimento. Joinville: UniSENAI, 2026.